

To keep the workload perfectly balanced, we need to look at **Technical Complexity** and **Research Time**.

The “Ears” (Student A) has the challenge of hardware and library setup, while the “Hands” (Student B) has the challenge of system logic and command execution. Here is how to split the tasks so neither of you feels overwhelmed or bored.

Balanced Workload Breakdown

Student A: The Integration Specialist (Ears)

- **Difficulty:** High initial setup, but easier once the library is connected.
- **Task 1: Environment & Model.** You are responsible for downloading the Vosk model and configuring the project libraries. You’ll be the “expert” on why the program is or isn’t hearing sound.
- **Task 2: The VoiceListener Object.** You write the code that opens the microphone and feeds sound to the Vosk “Brain.”
- **Task 3: Text Sanitization.** You ensure the String coming out of the library is clean (removing extra spaces or punctuation) so Student B can read it easily.

Student B: The Logic Specialist (Hands)

- **Difficulty:** Low initial setup, but high complexity in making sure the commands work across different situations.
 - **Task 1: The ActionEngine Object.** You write the logic that interprets the text. You have to handle “Synonyms” (e.g., if the user says “Launch Notepad” or “Open Notepad,” the program should do the same thing).
 - **Task 2: System Execution.** You research the specific Java commands to open apps like Chrome or Calculator and handle “Exceptions” (errors) if an app fails to open.
 - **Task 3: File I/O Feature.** You write the specific class that lets the user say “Save Note” to create a .txt file on the desktop.
-

Comparison of Workload

Factor	Student A (Ears)	Student B (Hands)
Research	Finding how libraries work.	Finding how OS commands work.

Factor	Student A (Ears)	Student B (Hands)
Coding Volume	~50-70 lines of setup code.	~60-80 lines of logic code.
Logic	Data stream management.	Decision-making & String matching.
Testing	Testing different microphones.	Testing different app paths.

The “Fairness” Protocol (Follow this strictly!)

To ensure no one does “more” or “easier” work, use these three rules:

1. **The Pair-Programming Kickoff:** Sit together for the first **1 hour**. Together, you will create the Main.java file and define the empty methods. This ensures you both know the “skeleton” of the project.
2. **The Bug Swap:** If Student A gets stuck on a microphone error for more than 30 minutes, Student B **must** stop and help. If Student B can’t get Chrome to open, Student A **must** help.
3. **The “Equal Features” Rule:** * Student A is responsible for the “**Open Browser**” command.
 - o Student B is responsible for the “**Open Notepad**” command.
 - o This way, you both learn how to use ProcessBuilder (the tool that opens apps).

Final Verdict on Difficulty:

Student A’s work is **conceptually** harder (audio streams can be confusing), but Student B’s work is **logically** longer (writing all the different cases for different commands). By the time you finish, you will both have spent roughly the same amount of hours on the project.

Does this division look like a 50/50 split to you, or would you like to swap one of the tasks?